

Lab 2 – Conversion from Regular Expression to NFA

Aim

To construct a Non-deterministic Finite Automaton (NFA) from a given regular expression using Thompson's construction method.

Algorithm

1. Start the program.
2. Read the regular expression in **postfix form**.
3. Use a stack to store partial NFAs.
4. Scan the expression symbol by symbol.
5. If the symbol is an operand:
 1. Create an NFA with two states.
 2. Add a transition using that symbol.
 3. Push the NFA onto the stack.
6. If the symbol is **concatenation (.)**:
 1. Pop two NFAs.
 2. Connect final state of first NFA to start state of second NFA using **ϵ -transition**.
 3. Push resulting NFA.
7. If the symbol is **union (|)**:
 1. Pop two NFAs.
 2. Create new start and final states.
 3. Add ϵ -transitions from new start to both NFAs.
 4. Add ϵ -transitions from both final states to new final state.
 5. Push resulting NFA.
8. If the symbol is **Kleene star (*)**:
 1. Pop one NFA.
 2. Create new start and final states.
 3. Add ϵ -transitions to allow repetition and empty string.
 4. Push resulting NFA.
9. After scanning completes, the remaining NFA on stack is the final NFA.
10. Display the transitions.
11. Stop.

C Program

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100
```

```

typedef struct {
    int from;
    int to;
    char symbol;
} Transition;

typedef struct {
    int start;
    int end;
} NFA;

Transition trans[MAX];
NFA stack[MAX];

int top = -1;
int state = 0;
int tcount = 0;

void push(NFA n) {
    stack[++top] = n;
}

NFA pop() {
    return stack[top--];
}

void addTransition(int from, int to, char symbol) {
    trans[tcount].from = from;
    trans[tcount].to = to;
    trans[tcount].symbol = symbol;
    tcount++;
}

void symbolNFA(char c) {
    NFA n;
    n.start = state++;
    n.end = state++;
    addTransition(n.start, n.end, c);
    push(n);
}

void concatNFA() {
    NFA n2 = pop();
    NFA n1 = pop();
    addTransition(n1.end, n2.start, 'e');
    NFA n;
    n.start = n1.start;
    n.end = n2.end;
    push(n);
}

```

```

}

void unionNFA() {
    NFA n2 = pop();
    NFA n1 = pop();

    NFA n;
    n.start = state++;
    n.end = state++;

    addTransition(n.start, n1.start, 'e');
    addTransition(n.start, n2.start, 'e');
    addTransition(n1.end, n.end, 'e');
    addTransition(n2.end, n.end, 'e');

    push(n);
}

void starNFA() {
    NFA n1 = pop();

    NFA n;
    n.start = state++;
    n.end = state++;

    addTransition(n.start, n1.start, 'e');
    addTransition(n.start, n.end, 'e');
    addTransition(n1.end, n1.start, 'e');
    addTransition(n1.end, n.end, 'e');

    push(n);
}

int main() {
    char postfix[100];
    int i = 0;

    printf("Enter postfix regular expression: ");
    scanf("%s", postfix);

    while (postfix[i] != '\0') {
        char c = postfix[i];

        if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
            symbolNFA(c);
        }
        else if (c == '.') {
            concatNFA();
        }
        else if (c == '|') {

```

```

        unionNFA();
    }
    else if (c == '*') {
        starNFA();
    }

    i++;
}

NFA result = pop();

printf("\nNFA Transitions:\n");
for (i = 0; i < tcount; i++) {
    printf("q%d -- %c --> q%d\n",
        trans[i].from,
        trans[i].symbol,
        trans[i].to);
}

printf("\nStart State : q%d\n", result.start);
printf("Final State : q%d\n", result.end);

return 0;
}

```

Sample Input

ab.

Sample Output

NFA Transitions:

```

q0 -- a --> q1
q2 -- b --> q3
q1 -- e --> q2

```

Start State : q0

Final State : q3

Result

Thus the **conversion of regular expression to NFA** was implemented successfully using **Thompson's construction method**.